

PPO + Offline BC 冲突问题总结报告

1. 当前问题背景

你的系统目前有两类策略来源：

online PPO policy：

在类别 A 上很强

在类别 B 上较弱

offline heuristic / teacher：

在类别 B 上很强

在类别 A 上较弱

你现在尝试把 offline teacher 数据通过 BC 注入训练。实验中发现：

$\cos(\text{grad_PPO}, \text{grad_BC}) \approx -0.3$

这说明 PPO objective 和 BC objective 在参数空间里存在明显冲突。这个负相似度不是小噪声，而是说明：

PPO 想把 policy 往一个方向推

BC 想把 policy 往另一个方向推

由于 teacher 并不是全局优于 PPO，而是只在某些 instance / state 上优于 PPO，所以无条件 BC 会产生负迁移。

2. 对当前训练方式的判断

你现在的训练方式是：

PPO 训练 5 epoch

然后做一次 offline BC

这个结构的问题是：

1. PPO 先把策略往 reward / objective 方向优化
2. BC 再把策略往 teacher 行为方向拉

3. 如果 PPO 和 BC 梯度冲突, BC 会抵消 PPO 的更新
4. 因为 BC 是最后一步, 所以最终 policy 可能偏向 teacher, 而不是 PPO objective

尤其是在 teacher 对 A 类表现差的情况下, 最后一次 BC 可能会把 PPO 在 A 类上学到的优势破坏掉。

所以问题不只是:

PPO 5 epoch 是否够

BC 1 次是否太多

而是:

BC 是否应该在所有 offline 样本上执行

BC 是否应该更新整个网络

BC 是否应该放在 PPO 之后

BC 的更新量是否受到约束

3. 核心结论

当前最关键的结论是:

不要把 offline teacher 当成全局专家。

teacher 应该被视为:

在部分 instance / state 上可信的局部专家

因此, offline 阶段不应该做普通 BC, 而应该做:

objective-filtered BC

也就是只在 teacher 明显优于当前 policy 的样本上做 BC。

4. 不需要在线知道 A/B 类别

一个重要约束是: 在实际跑一个新 instance 时, 你并不知道它属于 A 还是 B。

因此，不建议设计依赖显式 A/B 标签的方案：

```
if is_B:
    use teacher
else:
    use PPO
```

更合理的方式是给每个 offline 样本计算一个“teacher 可信度”：

```
[
w_i \in [0, 1]
]
```

它表示：

在这个 state / instance 上，teacher action 是否值得当前 policy 模仿

所以训练逻辑应该从：

这个样本是不是 B 类？

改成：

teacher 在这个样本上的 objective 是否明显优于当前 policy？

5. 你的 objective-filter 方向是对的

你目前正在做：

如果 teacher objective 更低，就使用这个例子
否则不用

因为你的最终目标是：

最小化总距离 / objective

所以 lower objective better。这个筛选方向是正确的。

但建议从：

```
use = teacher_obj < policy_obj
```

改成：

```
use = teacher_obj + margin < policy_obj
```

或者等价地：

```
[  
  \Delta_i = C_\pi(s_i) - C_T(s_i)  
]
```

其中：

$C_\pi(s_i)$ ：当前 policy 的 cost / objective 估计

$C_T(s_i)$ ：teacher 预计算的 cost / objective

teacher 更好意味着：

```
[  
  \Delta_i > 0  
]
```

但真正使用这个样本时，应该要求：

```
[  
  \Delta_i > margin  
]
```

也就是：

```
improve = obj_policy - obj_teacher  
use = improve > margin
```

这样可以避免 teacher 只是略微更好、或者估计误差导致的错误 BC。

6. 推荐的 offline BC loss

推荐将普通 BC：

```
[  
  L_{BC} = -\log \pi_\theta(a_T | s)  
]
```

改成 weighted / filtered BC：

[
 $L_{\{BC\}}$

$$-\frac{\sum_i w_i \log \pi_{\theta}(a_T^i | s_i)}{\sum_i w_i + \epsilon}$$

其中：

```
improve = obj_policy - obj_teacher # lower objective is better
margin = margin_abs + margin_rel * obj_policy.abs()
w = (improve > margin).float()
```

更平滑的版本：

```
raw = (improve - margin) / temperature
w = torch.sigmoid(raw)
w = w * (improve > margin).float()
w = w.clamp(max=w_max)
```

当前阶段建议先用 **hard mask**：

```
w = (improve > margin).float()
```

因为你现在最大的问题是负迁移，应该优先保证 teacher 样本的 precision，而不是 recall。

7. offline objective 最好使用 cost-to-go

如果 offline 数据是 trajectory，最好不要只用整条 episode 的最终 objective。

更推荐为每个 transition 存：

teacher 从当前 state 开始的 cost-to-go

即：

```
[  
C^T_t  
]
```

offline item 最好长这样：

```
offline_item = {  
    "state": s_t,  
    "teacher_action": a_t,  
    "teacher_cost_to_go": C_teacher_t,  
}
```

训练时比较：

```
improve = C_policy(s_t) - C_teacher_t
```

这比所有 transition 共享同一个 episode objective 更准确。

否则可能出现：

某个局部 action 本身不好

但整条 trajectory 后面被 teacher 修回来了

最终 objective 仍然不错

于是 BC 错误模仿了这个局部 action

8. offline 阶段不建议做 PPO

我们讨论后认为：offline 阶段读取 offline instance 后，不建议同时做 PPO 和 BC。

推荐：

offline 阶段：

只做 filtered / weighted BC

online 阶段：

正常做 PPO

原因是 PPO 是 on-policy 方法，它的 ratio / surrogate objective 依赖当前 policy 采样的数据。offline heuristic 数据不是当前 policy 采样出来的，如果直接拿 offline 数据算 PPO loss，很容易变成“假 PPO”，导致估计有偏。

所以 offline 数据应该用于：

1. filtered BC
2. 可选: 训练 teacher-cost predictor
3. 可选: 训练 gate / trust model

不建议用于:

直接计算 PPO actor loss

除非你决定切换到 AWAC、IQL、off-policy actor-critic 这类算法框架。

9. critic 和 policy 共享网络时的关键风险

你的 actor 和 critic share 一个网络结构, 这会带来一个非常重要的问题:

BC loss 不只会更新 actor

也会通过 shared trunk 改变 critic 的输入表征

如果网络是:

```
shared_encoder → actor_head  
shared_encoder → critic_head
```

那么 offline BC 如果更新整个网络, 会发生:

1. BC 改变 shared_encoder
2. actor 行为变化
3. critic 的 feature representation 也变化
4. critic_head 没有被正确重新适配
5. 下一轮 PPO 的 value / advantage 可能变脏

因此, 在 shared actor-critic 结构下, offline BC 不能无脑更新整个网络。

10. 推荐的网络更新策略

最低成本方案

offline BC 阶段只更新 actor head:

```
freeze shared_encoder
freeze critic_head
train actor_head
```

也就是：

```
set_requires_grad(shared_encoder, False)
set_requires_grad(critic_head, False)
set_requires_grad(actor_head, True)
```

offline BC 结束后，再恢复 PPO 正常训练：

```
set_requires_grad(shared_encoder, True)
set_requires_grad(critic_head, True)
set_requires_grad(actor_head, True)
```

更推荐的结构

如果 actor head 容量不足，可以加 actor-only adapter：

```
shared_encoder → actor_adapter → actor_head
shared_encoder → critic_head
```

offline BC 阶段只更新：

```
actor_adapter
actor_head
```

不更新：

```
shared_encoder
critic_head
```

这个结构的好处是：

1. BC 有足够容量调整 policy
2. 不破坏 critic representation
3. 不直接污染 PPO value / advantage

最理想结构

如果工程允许，最好把 actor encoder 和 critic encoder 分开：

```
actor_encoder → actor_head
critic_encoder → critic_head
```


这样 offline BC 只影响 actor，不会影响 critic。

11. 筛选时应该使用 frozen reference critic

由于 offline BC 可能改变网络，筛选权重不应该在 BC 更新过程中用 live critic 反复计算。

推荐流程：

offline BC 前，保存一个 frozen reference model
用 frozen critic / value 计算 policy objective
用这个结果决定哪些 offline 样本进入 BC

伪代码：

```
model_ref = deepcopy(model).eval()
freeze(model_ref)

with torch.no_grad():
    obj_policy_ref = cost_from_ref(model_ref, s_off)

improve = obj_policy_ref - obj_teacher
w = (improve > margin).float()
```

这样可以避免：

BC 改 shared trunk
critic estimate 漂移
mask / weight 跟着漂
BC 继续按被污染的 critic 筛选

12. 不建议用 teacher objective 训练 PPO critic

PPO critic 应该估计：

当前 policy 的 value / cost

而 teacher objective 表示：

teacher policy 的 cost-to-go

两者不是同一个函数。

因此不建议这样做：

```
ppo_critic(s_off) = teacher_objective
```

否则 PPO advantage 会被污染。

如果确实想学习 teacher objective，可以单独加一个 head：

```
ppo_value_head:
    只用 online rollout 训练

teacher_cost_head:
    用 offline teacher objective 训练
    只用于 filtering / gate / trust score
```

13. 推荐的训练顺序

当前顺序是：

```
collect rollout
PPO 5 epoch
offline BC
```

问题是最后一步是 BC，policy 最终更容易偏向 teacher。

更推荐的顺序是：

```
offline filtered BC
collect rollout
PPO 5 epoch
```

也就是每轮：

1. 用 frozen reference 计算 offline 样本筛选权重
2. offline filtered BC，只更新 actor head / actor adapter
3. 用 BC 后的 policy 收集 online rollout
4. PPO 5 epoch，正常更新 actor + critic

这样有三个好处：

1. rollout 是用 BC 后的当前 policy 采的，on-policy 更干净

2. 每轮最后一步是 PPO，最终策略仍然由 objective / reward 主导
3. critic 会在 BC 后的 policy 分布上重新训练

14. 推荐的训练循环

推荐版本：

```
for iteration in range(num_iterations):

    # -----
    # 1. frozen reference for filtering
    # -----
    model_ref = deepcopy(model).eval()
    freeze(model_ref)

    # -----
    # 2. offline filtered BC phase
    # -----
    freeze(model.shared_encoder)
    freeze(model.critic_head)
    train(model.actor_adapter)    # if exists
    train(model.actor_head)

    for _ in range(num_offline_steps):

        s_off, a_teacher, obj_teacher = sample_offline_batch()

        with torch.no_grad():
            obj_policy_ref = cost_from_ref(model_ref, s_off)

            # lower objective is better
            improve = obj_policy_ref - obj_teacher

            margin = margin_abs + margin_rel * obj_policy_ref.abs()

            w = (improve > margin).float()

            if w.sum() < min_selected:
                continue

            dist = model.policy(s_off)
            logp = dist.log_prob(a_teacher)

            if logp.ndim > 1:
                logp = logp.sum(dim=-1)

            loss_bc = -(w * logp).sum() / (w.sum() + 1e-8)

            optimizer_bc.zero_grad()
            loss_bc.backward()
            clip_grad_norm_(actor_bc_params, bc_max_norm)
            optimizer_bc.step()
```

```

# restore full model for PPO
train(model.shared_encoder)
train(model.actor_adapter)
train(model.actor_head)
train(model.critic_head)

# -----
# 3. collect rollout after BC
# -----
online_batch = collect_rollout(model)

# -----
# 4. PPO phase
# -----
for _ in range(5):
    for mb in online_loader:

        loss_ppo, loss_v, entropy = compute_ppo_loss(mb)

        loss = loss_ppo + vf_coef * loss_v - ent_coef * entropy

        optimizer_ppo.zero_grad()
        loss.backward()
        clip_grad_norm_(ppo_params, ppo_max_norm)
        optimizer_ppo.step()

```

15. offline 更新频率建议

不要用“offline 做几次 BC”作为主要标准，而应该看 offline update 的实际更新量。

定义：

$$\left[\frac{\Delta \theta_{\text{offline}}}{\Delta \theta_{\text{PPO}}} \right]$$

建议初始控制在：

```

rho_target = 0.03 ~ 0.10
rho_max    = 0.20

```

因为你已经观察到：

```
cos(PPO, BC) ≈ -0.3
```

所以建议从更保守的设置开始：

```
rho_target = 0.03 ~ 0.05
```

实际频率建议:

每轮 offline filtered BC: 1 个 minibatch 起步

稳定后增加到 2~4 个 minibatch

不要一开始做完整 offline epoch

如果 offline BC 仍然会更新 shared trunk, 则更保守:

每 5~10 个 PPO iteration 做一次 offline BC

每次只做 1 个 minibatch

$lr_bc \leq 0.01 \sim 0.05 * lr_ppo$

但更推荐直接冻结 shared trunk。

16. learning rate 和 beta 建议

如果 offline BC 只更新 actor head / adapter:

$lr_bc \approx 0.05 \sim 0.1 * lr_ppo$

如果仍然更新 shared trunk:

$lr_bc \leq 0.01 \sim 0.05 * lr_ppo$

BC loss coefficient:

beta 从 0.01 ~ 0.1 起步

不要一开始设置太大。因为 teacher 不是全局专家, BC 不能和 PPO 平权。

17. margin 设置建议

如果 objective 已经 normalize, 可以试:

$margin_abs = 0.05 \sim 0.10 * objective_std$

$margin_rel = 0.02 \sim 0.10$

如果 objective scale 不稳定, 建议先做 running normalization:

```
obj_norm = (obj - running_mean) / (running_std + 1e-8)
```

然后在 `normalized objective` 上设 `margin`。

对于最小化任务，筛选逻辑统一为：

```
improve = policy_cost - teacher_cost  
use = improve > margin
```

如果 PPO critic 输出的是 `reward value`，而 `reward` 约等于 `-distance`，要注意符号：

```
policy_cost = -value_ref(s)  
improve = policy_cost - teacher_cost
```

不要直接拿 `reward value` 和 `cost objective` 比。

18. 是否需要 KL anchor

如果保持旧顺序：

```
collect rollout  
PPO 5 epoch  
offline BC
```

那么强烈建议加 KL anchor：

[
 L_{offline}

$$\begin{aligned} & \beta L_{\text{filtered_BC}} \\ & + \\ & \lambda D_{\text{KL}}(\pi_{\text{ref}}, \pi_{\theta}) \end{aligned}$$

其中 π_{ref} 是 PPO 后、BC 前的 policy。

目的：

允许 BC 在 teacher 明显更好的样本上调整 policy
但限制 BC 大幅破坏 PPO 刚学好的行为

如果改成推荐顺序：

```
offline BC
collect rollout
PPO
```

KL anchor 不是必须，但仍然可以作为保护项。

19. 需要记录的诊断指标

后续分析时建议每轮记录这些指标。

数据筛选指标

```
selected_ratio = mean(w > 0)
mean(improve)
mean(margin)
mean(improve | w > 0)
offline_bc_loss
```

用于判断 **teacher** 样本到底有多少被使用。

梯度冲突指标

```
cos(PPO_grad, raw_BC_grad)
cos(PPO_grad, filtered_BC_grad)
```

理想情况：

```
raw_BC_grad 可能仍然是 -0.3
filtered_BC_grad 应该接近 0 或变正
```

如果 **filtered BC** 后 **cosine** 仍然强负，说明筛选不够严格。

更新量指标

```
||Δoffline|| / ||ΔPPO||
```

目标：

0.03 ~ 0.10 起步
不要长期超过 0.20

KL 指标

```
KL(policy_after_BC || policy_before_BC) on online states  
KL(policy_after_PPO || policy_before_PPO)
```

如果 BC 后 KL 很大，说明 offline update 太强。

critic 稳定性指标

尤其因为 actor 和 critic share 网络，需要记录：

```
critic_loss_before_BC  
critic_loss_after_BC  
value_prediction_drift_after_BC  
advantage_mean / std
```

如果 BC 后 critic loss 或 value drift 明显上升，说明 shared representation 被污染。

objective 分桶指标

即使在线不知道 A/B，离线分析仍然可以按 bucket 看：

```
A-like bucket objective  
B-like bucket objective  
overall objective  
teacher-selected samples objective  
teacher-rejected samples objective
```

目标是：

```
A-like 不掉  
B-like 提升  
overall 下降
```

20. 可能的扩展方案

方案一：actor adapter

这是当前最推荐的工程改动：

```
shared_encoder → actor_adapter → actor_head  
shared_encoder → critic_head
```

offline BC 只更新：

```
actor_adapter + actor_head
```

PPO 更新全部。

方案二：单独 teacher-cost head

如果 teacher objective 需要泛化到未见 state，可以加：

```
teacher_cost_head
```

它只用于：

```
filtering  
trust score  
gate
```

不要混入 PPO critic。

方案三：mixture-of-experts

如果后续发现 A/B 差异非常强，单 actor 难以兼容，可以考虑：

```
actor_A: PPO strong expert  
actor_B: teacher / BC strong expert  
gate: 判断当前更应该使用哪个 expert
```

但由于你当前在线不知道 A/B，这个 gate 不能依赖真实标签，而应该学习：

```
P(teacher better than current policy | state/context)
```

也就是用 objective improvement 作为监督信号。

方案四：历史信息 / RNN

如果当前 observation 本身无法判断 teacher 是否可信，则单步 state policy 可能不够。

这时需要考虑：

```
history encoder
RNN / GRU
past k steps context
task embedding
环境参数 embedding
```

因为如果 A/B 的区分信息不在当前 observation 中，policy 或 gate 都无法可靠判断该走哪种行为模式。

21. 当前最推荐的落地方案

综合所有讨论，当前优先级最高的方案是：

1. offline 不做 PPO，只做 filtered BC
2. teacher objective 必须比 current policy objective 低超过 margin 才使用
3. offline BC 只更新 actor head / actor adapter
4. freeze shared encoder 和 critic head
5. offline BC 放在 rollout 前，PPO 放在最后
6. offline 更新频率从 1 个 minibatch / iteration 开始
7. 监控 $||\Delta_{\text{offline}}||$ / $||\Delta_{\text{PPO}}||$ ，控制在 0.03~0.10
8. 记录 filtered BC 后的 gradient cosine

推荐训练顺序：

for each iteration:

```
frozen reference model

offline objective-filtered BC
  use sample only if:
    policy_cost - teacher_cost > margin
  update:
    actor_head / actor_adapter only
  freeze:
    shared_encoder
    critic_head

collect online rollout with updated policy
```

PPO 5 epochs
update actor + critic + shared network

22. 一句话总结

你当前的问题本质是：

offline teacher 是局部专家，不是全局专家；
普通 BC 把局部专家当成全局专家，导致和 PPO 梯度冲突。

后续应该把 offline 阶段改成：

有 margin 的 objective-filtered BC

并且由于 actor / critic 共享网络，BC 阶段必须隔离梯度：

只更新 actor head / actor adapter

不更新 shared trunk

不更新 critic

最终目标不是“尽可能多地使用 offline 数据”，而是：

只在 teacher 明显优于当前 policy 的地方使用 offline 数据；
在其他地方让 PPO 自己优化。